

Notes on the Monte Carlo Line Transfer

BY LILE WANG

1 General and Basic Information

1.1 Conventions

On the Kratos side, everything with the suffix "_cgs" is in CGS, and in code unit otherwise (the quantities on the Kratos side should be predominantly in the code unit system).

The unit system (or dimension, as in the term "dimensional analyses") in this note is presented with the bracked lower-case letters in math, the dimension of length using $[l]$, time using $[t]$, mass in $[m]$. For example, the acceleration has unit $[l][t]^{-2}$. Specifically, although the photon number does not have units, we can still mark it with a special symbol $[n]$ to "pretend" that it has a specific dimension of "counting", which can be equivalent to dimensionless once necessary.

In this note, we typically refer to "wavelength" or "frequency" of photons as "velocity"—the shift of the photon in the velocity space defined by the Doppler effect ($\Delta v = \Delta\lambda/\lambda_0 = -\Delta\nu/\nu_0$, $\Delta v > 0$ for redshifts, or longer wavelengths, compared to the stationary wavelength λ_0).

1.2 Photon packets

Each photon packet represents many photons evolved in unit time period, carries a lot of "photons per unit time" ("effective luminosity", equivalently, or "luminosity" for short). A photon packet assumes Gaussian distribution in the velocity space for its own "proper" photons, described by:

- $\mathcal{L}$, the total photon number per unit time ("luminosity") in the packet, described by the photon packet's "proper".
- Δv , the velocity-space centroid of the Gaussian distribution of the current photon packet. Note that this is a scalar, indicating the extent of wavelength shifts.
- σ_{ph} , the dispersion of the Gaussian profile, which has the same unit as velocity.
- $\hat{d}$, the normalized direction vector for the propagation direction of the photon packet.
- $\mathbf{x}$, the current spatial location vector of the photon packet.
- Other necessary information for computations (for example, the block index of the mesh on which the photon packet is propagating, and the cell index in the mesh-block).

The per-unit-time-period photon number ("luminosity") per unit velocity in the velocity space, for the photon packet, is described as,

$$\frac{\delta\mathcal{L}}{\delta v} = \frac{\mathcal{L}}{\sqrt{2\pi}\sigma_{\text{ph}}} \exp\left[-\frac{(v - \Delta v)^2}{2\sigma_{\text{ph}}^2}\right]. \quad (1)$$

1.3 Physical picture

In one sentence: "Optically-thick radiative transfer of a line".

We use the term "line" for the radiation phenomena related to one specific transition of the target particles (molecules, atoms, ions, and even electrons if working on inverse Compton scattering—we will use "particles" for a generic indication), as in "emission line" and "absorption line".

The system involves the following physical processes and mechanisms in the designated spatial region.

1.3.1 "Scattering"

There are target particles that are involved in the absorption of photons of the designated line (by the lower-state particles of the transition), and the subsequent re-emission (from the higher state). In steady states that this code aims for, the re-emission takes place immediately after the absorption relative to the problem timescales, thus this process can be treated as scatterings equivalently. Note that:

- These methods for the equivalent scattering can be directly used for "real" scattering, therefore we do not distinguish these "scatterings" mostly. There is one exception, though:
 - When re-emitted in an equivalent scattering event, the "new" (outgoing) photon packet should have almost isotropic distribution of direction of propagation, and the
 - When actually scattered, the outgoing photon packet should follow the energy and momentum conservation relations according to the
- Only the particles on the lower level of the transition can take part in these scatterings. Therefore, the population number of particles could be important.

1.3.2 "Absorption"

These are the "real" absorption processes that remove photons entirely from the target transition of target particles. In Monte Carlo methods, this leads to the deduction of the photon numbers in photon packets. For most line transfer, such "real" absorption could be treated as independent of Δv (equivalently, wavelength-independent).

1.3.3 Population numbers of the target particles

For the target line, the number densities of the particles on the lower and upper levels respectively at each spatial location are of core importance. Calculations of the population numbers of the concerned levels are carried out on the Python side, not the Kratos side. Note that there could be more levels than two to deal with (although only two are directly involved in the concerned line transition) to get the population numbers correct.

These calculations involve:

- Collisional excitation and de-excitation. This will require collision partner number density profiles, and one of the collisional (de-)excitation rate coefficients (the other could be calculated using the detailed balance principle)—multiplying the coefficients (dimension $[L]^3[t]^{-1}$) by the collision partner number density (dimension $[L]^{-3}$) will get us the (de-)excitation rates in transition probability per unit time (dimension $[t]^{-1}$).
- Spontaneous decay. This will require the Einstein A coefficients (dimension $[t]^{-1}$) of the relevant levels to other levels. These could result in the deduction of numbers in many "source" levels, and add to the numbers in the levels as the destination of the decay (note that these may or may not be the two levels involved in the line transition—take extra care of the level symbols).

- Photon excitation. The Kratos output will include an "effective excitation flux" F_{ext} (already taking the Gaussian profiles of the photon packet and thermal broadening into account, see later), in photon number per unit area per unit time (dimension $[n][l]^{-2}[t]^{-1}$), which can be multiplied by the line-center excitation cross section σ_0 (dimension $[l]^2$) of excitation to get the photon excitation rate in transition probability per unit time (dimension $[t]^{-1}$). Note that the normalized Gaussian profile has already been involved in obtaining F_{ext} , so that the product $F_{\text{ext}}\sigma_0$ will get the photon excitation rate directly. Note also that it is very likely that the excitation flux is involved **only** in the target transition. Additionally, these total cross sections of absorption from lower level to higher level, can be calculated via the Einstein A coefficient of spontaneous decay from the upper level to the lower level (see the next sub-subsection).

One simplification that could be adopted in many situations is that the system is a two-level system, and only the lower level and the upper level of the target transition are involved. Nevertheless, we should guarantee the ability to calculate systems with many levels (but only two of them are involved in the Monte Carlo radiative transfer).

1.3.4 Scattering cross sections

The definition of the line-center excitation cross section σ_0 also deserves some discussions. Ignoring the intrinsic width, the cross section at velocity v approximately reads, under the thermal broadening $\sigma_{\text{th}} = \sqrt{k_B T / \mu} = 9.12 \text{ km s}^{-1} \times [T_4 / (\mu / m_A)]^{1/2}$ (k_B the Boltzmann constant, μ the dimensional particle mass, $T_4 \equiv T / (10^4 \text{ K})$, and m_A the atomic mass unit) in the particle-rest frame (eq. 6.39 in Draine 2011),

$$\sigma(v) = \frac{\sqrt{\pi} e^2}{m_e c} \frac{f \lambda_0}{\sqrt{2} \sigma_{\text{th}}} \exp\left(-\frac{v^2}{2\sigma_{\text{th}}^2}\right), \quad (2)$$

where $e = 4.80321 \times 10^{-10} \text{ g}^{1/2} \text{ cm}^{3/2} \text{ s}^{-1}$ is the CGS electron charge, m_e the mass of electron, c the speed of light, λ_0 the line-center wavelength, and the dimensionless oscillator strength f is related to the Einstein A coefficient by (eq. 6.20 in Draine 2011),

$$A = \frac{8\pi^2 e^2 \nu^2}{m_e c^3} \frac{g_l}{g_u} f = \frac{0.6670 \text{ cm}^2 \text{ s}^{-1}}{\lambda_0^2} \frac{g_l}{g_u} f. \quad (3)$$

where $\nu_0 = c / \lambda_0$ is the line-center frequency, g_l and g_u the degree of degeneracy of the lower and upper levels ($g = 2J + 1$ for each level). We can re-write the $\sigma(v)$ expression as $\sigma(v) = \sigma_0 \exp[-v^2 / (2\sigma_{\text{th}}^2)]$, where,

$$\sigma_0 = \frac{\sqrt{\pi} e^2}{m_e c} \frac{f \lambda_0}{\sqrt{2} \sigma_{\text{th}}}. \quad (4)$$

In practice, the Python-side pipeline will calculate the oscillator strength f based on the Einstein A coefficient and the degeneracy g_u and g_l based on the J of the corresponding levels, by looking for the data in the database. Then, the Python-side pipeline will then calculate this σ_0 field based on the temperature distribution in the space, and then calculate the $\lambda_{\text{sca},0}^{-1}$ (reciprocal of the line-center scattering mean free path) by $\lambda_{\text{sca},0}^{-1} \equiv \sigma_0 n_{\text{lower}}$ (n_{lower} the number density of the target particle on the lower state of the target transition), feed this as one of the input fields to Kratos, within the binary file that contain all necessary fields to be read by Kratos.

2 Methods on the Kratos side

One sentence guideline: Instead of directly emulating the particles and photons, Monte Carlo methods sample the differential equations that the photons should obey.

2.1 Photon packet proper

For each photon packet, the "proper" is actually used as the effective luminosity $\mathcal{L}$, in units of $[n][t]^{-1}$. When it passes through a cell with volume V using a intra-cell path length δl (measured from the entry point into the current cell through the exit point of the cell), this packet contributes to the flux (photon number flux, not the energy flux, here and in what follows) in the cell by $\delta F = \mathcal{L} \delta l / V$ ($V / \delta l$ is the effective surface area of this packet crossing the current cell), and the final number of the flux in that cell should be calculated by summing all the δF contributed by all photons crossing the cell. It is natural to see that the dimension of δF is $[n][l]^{-2}[t]^{-1}$.

2.2 Excitation

The excitation flux contributed by one ray, δF_{exc} , is just the flux contribution δF multiplied by the dimensionless overlap integral $\mathcal{I}$ ($\delta F_{\text{exc}} = \mathcal{I} \delta F$), where $\mathcal{I}$ is defined as,

$$\begin{aligned} \mathcal{I} &= \frac{1}{\sqrt{2\pi\sigma_{\text{ph}}^2}} \int_{-\infty}^{+\infty} dv' \exp\left[-\frac{(v' - \Delta v)^2}{2\sigma_{\text{ph}}^2}\right] \exp\left[-\frac{(v' + v_{\parallel})^2}{2\sigma_{\text{th}}^2}\right] \\ &= \frac{1}{\sqrt{1 + \sigma_{\text{ph}}^2/\sigma_{\text{th}}^2}} \exp\left[-\frac{(\Delta v + v_{\parallel})^2}{2(\sigma_{\text{ph}}^2 + \sigma_{\text{th}}^2)}\right], \end{aligned}$$

where σ_{ph} and Δv are the dispersion and line-center velocity ($\Delta v > 0$ for redshifts) for the Gaussian of the photon packet (one of the member data of the photon packet), σ_{th} is the thermal-motion Gaussian dispersion of the gas (calculated by the temperature and the), $v_{\parallel} = \mathbf{v} \cdot \hat{\mathbf{d}}$ is the bulk motion velocity of the gas at the current location (denoted by $\mathbf{v}$, the data fields that Kratos reads in at the beginning of each cycle) projected along the propagation direction of the photon. Summing up the δF_{ext} by all photons passing through the current cell in the Monte Carlo, one gets the F_{ext} of the current cell. The Python side will read this F_{ext} from the Kratos output, and use $F_{\text{ext}}\sigma_0$ to calculate the population number in preparation for the next cycle.

2.3 Evolution

Again, in this work, "scattering" has a special denotation: absorbed by the lower-level and re-emitted instantly by the upper level (already described in the previous section).

2.3.1 Propagation

The same expression of overlap integral $\mathcal{I}$ is also used in determining the scattering event location. After each scattering event (or after the photon is generated or read into Kratos, marked as the "0th" scattering), a "remaining scattering optical depth" τ_{rem} is generated following exponential distribution of parameter 1, which is easy: first generate a uniform distribution $Y \in [0.0001, 0.9999]$ (in theory, $Y \in [0, 1]$, but we use $[0.0001, 0.9999]$ for safety), then assign $\tau_{\text{rem}} = -\ln Y$.

The photon packet then travels in a straight line along its direction vector $\hat{\mathbf{d}}$, from surface to surface of cells. When reaching the exit surface of the current cell (the photon gets its exit point) after travelling for δl in the current cell, the photon gets a $\delta\tau_{\text{rem}} \equiv \mathcal{I} \lambda_{\text{sca},0}^{-1} \delta l$, then the updated value τ'_{rem} is obtained by $\tau'_{\text{rem}} \leftarrow \tau_{\text{rem}} - \delta\tau_{\text{rem}}$ assuming that the photon will reach the cell-exiting point.

Now, if $\tau'_{\text{rem}} > 0$, then the photon packet continues propagation along $\hat{\mathbf{d}}$. The absorption is taken into account by updating the photon packet's proper, $\mathcal{L}' \leftarrow \mathcal{L} \exp(-\delta l \lambda_{\text{abs}}^{-1})$, where $\lambda_{\text{abs}}^{-1}$ is the inverse (reciprocal) of the ("real") absorption mean free path passed into Kratos by the Python pipeline.

If $\tau'_{\text{rem}} < 0$, an linear interpolation is carried out to find the location at which the scattering takes place: $\mathbf{x} = \mathbf{x}_{\text{enter}} + (\tau_{\text{rem}} / \delta\tau_{\text{rem}}) \hat{\mathbf{d}} \delta l$ ($\mathbf{x}_{\text{enter}}$ is the spatial coordinates that the photon packet enters the current cell; $\mathbf{x}_{\text{exit}} = \mathbf{x}_{\text{enter}} + \hat{\mathbf{d}} \delta l$ gives the location that the photon packet exits the cell if the scattering event does not take place in the current cell). The absorption is also taken into account, by $\mathcal{L}' \leftarrow \mathcal{L} \exp[-(\tau_{\text{rem}} / \delta\tau_{\text{rem}}) \delta l \lambda_{\text{abs}}^{-1}]$ (also linear interpolation in the absorption optical depth).

2.3.2 Scattering event

When a scattering event takes place, we must re-assign the direction $\hat{d}$, the broadening σ_{ph} , and the velocity shift Δv , to the scattered photon packet. Because we assume that all scatterings are "quick re-emission after absorption", we can use a simplified scheme ignoring the particle recoil:

1. Assign $\hat{d}$ by generating $\mu \equiv \cos\theta \in [-1, 1]$ and $\varphi \in [0, 2\pi]$, obeying uniform distributions, respectively, and set $\hat{d} = [\sin\theta \cos\varphi, \sin\theta \sin\varphi, \cos\theta]$.
2. Assign $\Delta v = -\mathbf{v} \cdot \hat{d}$ according to the bulk motion velocity $\mathbf{v}$ of the current cell. Note the sign: when $\mathbf{v}$ and $\hat{d}$ are in the same direction, the photon packet is blue shifted ($\Delta v < 0$).
3. Assign $\sigma_{\text{ph}} = \sigma_{\text{th}}$ according to the σ_{th} of the current cell.
4. Generate a new τ_{rem} in the same way as the previous sub-subsection.

3 Methods on the Python side

The python-side jobs should be relatively straightforward.

3.1 Data on mesh

3.1.1 To Kratos

The first is to provide the data for the Kratos side, via a binary file that includes the following fields on structured mesh:

- The inverse (reciprocal) of the effective line-center mean free path of scattering (excitation), $\lambda_{\text{sca},0}^{-1}$.
- The inverse (reciprocal) of the absorption $\lambda_{\text{abs}}^{-1}$.
- The thermal broadening velocity σ_{th} .
- The bulk motion velocity field $\mathbf{v}$.

These values shall be read into Kratos, used in the construction of interpolation objects, and set the mesh-block data by calling the interpolation fields. The Python interface will accept either the numbers (for initializing uniform fields) or callables (for initializing spatial-coordinate-dependent fields) for these fields, all in CGS (including dependent and independent variables). Specifically, the Python pipeline should also allow the user to specify the species, upper and lower energy levels, number density, and temperature of the concerned target particles, so that the Python pipeline will calculate $\lambda_{\text{sca},0}^{-1}$ and σ_{th} on its own. Also, the Python object should hold these data objects so that the user can plot these fields easily.

Note that all these quantities should be in the code unit system (including the independent variables, esp. the coordinates that are used in constructing the interpolation objects), converted from other unit systems (esp. CGS, presumably the unit system used on the Python side) by the Python-side routines (that is, conversion should be finished before feeding the data to Kratos). The unit system should be specified under the [unit] section in the parameter file to be read by Kratos.

3.1.2 From Kratos

The Python pipeline will read the F_{ext} from the Kratos output (be careful about the units conversion from the code unit to the CGS) and calculate the $F_{\text{ext}}\sigma_0$ for the photon excitation rate, as one of the key data used in the calculation of the population of particle energy levels.

3.2 Particle data

3.2.1 To Kratos

The Python side will generate photon packets to be read into Kratos, which consist of two parts:

- "External" source(s). Photons that are emitted by sources **other than** the target particles distributed in the space. For example, an isotropic point source for a star, or a plane-parallel extended source if one studies a photodissociation region (PDR).
- "Internal" sources. Each cell with particles on the upper level of the target transition is equivalent to an isotropic point source, whose luminosity is equivalent to the volume of the cell multiplied by the number density of the upper-level particles times the Einstein A coefficient. There must be at least one photon packet generated per cell, and at most $N_{\text{ph,cell,max}}$ photons generated per cell; the actual number of photon packets should be determined according to the cell's luminosity. The sum of the photons' effective luminosity should equal to the total luminosity of the cell.

We should take care of the units, paying attention to these things:

- Each photon packet carries an "equivalent luminosity", which is in photon-number luminosity dimension $([n][t]^{-1})$.
 - If dealing with a plane-parallel extended source style, the user shall specify the photon direction d and the photon number flux (dimension $[n][l]^{-2}[t]^{-1}$) of the plane parallel extended source; photon packets' initial location should guarantee that the sum of photon packets luminosity **per unit area perpendicular to the direction of all photon packets** matches the user-designated number.
 - If dealing with an isotropic point source, the user shall specify the photon number luminosity (dimension $[n][t]^{-1}$) of the point source. The photon packet directions $\hat{d}$ go through the all- 4π solid angle uniformly (could be generated by uniform distributions for $\cos\theta \in [-1, 1]$ and $\varphi \in [0, 2\pi]$), and the sum of effective luminosity of all photon packets generated should equal to the luminosity of the source.
- The Python-side routines work on the CGS system, while Kratos works on its own code unit system. Make the conversion correctly based on the code unit conversion ([unit] section in the parameter file for Kratos).

3.2.2 From Kratos

The Python routine will read the particles after the final iteration is done, in order to generate the statistics as the output.

3.3 Iterations

The calculations are calculated iteratively. In each iteration, we conduct steps:

1. On Python side: Calculate the population number of all relevant energy levels of the target particle using the temperature, collisional partner density, collisional (de-)excitation rates (calculated as functions of temperatures), excitation photon fluxes F_{exc} (and the line-center excitation cross section σ_0 , also as a function of temperature) . If this is the first iteration, ignore that the photon excitation (by setting $F_{\text{exc}} = 0$ everywhere) as we don't have F_{exc} ; otherwise, read F_{exc} from the Kratos output of the previous iteration.
2. On Python side: Create data cubes of $\lambda_{\text{abs}}^{-1}$, $\lambda_{\text{sca},0}^{-1}$, $\mathbf{v}$, σ_{th} (pay attention to units!), save them in a binary file, to be read by Kratos.

3. On Python side: Generate photon packets, save them in another binary file, to be read by Kratos
4. On Kratos side: Conduct Monte Carlo calculations, saving F (photon number flux) and F_{exc} (effective excitation photon number flux) and the final states of photons.

These steps are conducted iteratively until the system converges (or certain number of iterations is reached).